



DateTime

🕒 Created time	July 9, 2026 5:27 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 10, 2026 4:27 AM

```
using System.Globalization;

DateTime today = DateTime.UtcNow;

DateTime birthday = DateTime.Parse("15/06/1998");

DateTime birthday = DateTime.ParseExact("11/15/1998", "M/d/yyyy", Culture
Info.InvariantCulture);

DateTime birthday = DateTime.ParseExact("06/11/1998", "d/M/yyyy", Culture
Info.InvariantCulture);

Console.WriteLine(birthday.ToString());

Console.WriteLine(today.ToString("MMMM dd, yyyy hh:mm tt zzz", CultureInf
o.InvariantCulture));
```

```
DateTime today = DateTime.Now;
Console.WriteLine(today);
Console.WriteLine(today.ToString());
```

המשתנה **DateTime** קיים מאז תחילתה של #C.

ישנן אפשרויות ניתוח (**parsing**) חדשות יותר.

האובייקט **DateTime** הוא הדרך הנפוצה ביותר לאחסן מידע על תאריך ושעה.

לדוגמה:

```
DateTime today = DateTime.Now;
```

זה הולך להיות התאריך והשעה של הרגע הנוכחי וזה יוכנס למשתנה **today**.

זהו הזמן המקומי, המבוסס על המיקום של המחשב שלנו.

יש הרבה מורכבות סביב תאריך ושעה.

הרעיון כאן הוא שתאריך ושעה הם מורכבים מכיוון שהם פוטנציאלית שונים עבור כל אדם.

לדוגמה, אנחנו באזור זמן מרכזי (**Central Time**), ולכן 13:30 שלנו הוא למעשה 14:30 באזור זמן מזרחי (**Eastern Time**), בדיוק באותו הרגע.

אז יש כאן מורכבות עם אזורי זמן.

יש גם דברים כמו שעון קיץ, כאשר האזור שלנו מכבד שעון קיץ, ויש לנו בפועל שעון קיץ, אבל אזורים אחרים, אפילו בארה"ב, לא מכבדים שעון קיץ, ולכן בהתאם לתקופה בשנה יהיו זמנים שונים.

אז יש הרבה מורכבות סביב תאריך ושעה, יש גם שנה מעוברת וכל שאר הדברים שקורים.

אבל האובייקט **DateTime** מאחסן תאריך ושעה מלאים.

בדוגמה אנחנו מדפיסים את זה:

```
Console.WriteLine(today);
```

ומאחורי הקלעים זה מדפיס למחרוזת, ככה:

```
Console.WriteLine(today.ToString());
```

אבל בהדפסה הזו, יש לנו המון אפשרויות שם,

ל-ToString הזה יש הרבה אפשרויות, לשים מירכאות בתוך הסוגריים, ואנחנו יכולים לציון פורמט על ידי פתיחת רשימה נפתחת כמו:

```
long date,  
full short date/time ,  
full long date/time ,  
general short date/time
```

וכן הלאה.

לדוגמה אנחנו רוצים רק את התאריך, אז האות **d** הקטנה, אבל המשתנה עדיין מחזיק את אותו המידע, זה לא שינה את המידע שיש בתוך המשתנה, מה שזה שינה זה את מה שמוצג, מה שמודפס.

ואם אנחנו רוצים רק את השעה, אז האות **t** הקטנה היא ללא שניות, האות **T** הגדולה היא עם שניות וכן הלאה.

אז אנחנו יכולים להשתמש בגרסאות השונות האלה כדי להדפיס רק את החלק שאנחנו רוצים.

במשך זמן רב זה היה אובייקט התאריך או השעה היחיד שהיה לנו ב-**#C**, ולכן אם היינו רוצים לייצג את יום ההולדת שלנו, אז היה לנו **DateTime** אבל לא היינו דואגים לחלק של השעה, ובאופן מסורתי מה שהיינו עושים זה להגדיר את השעה לחצות, וכך התמודדנו עם תאריך בלבד.

ואם הייתה לנו רק שעה, למשל השעה שבה העסק נפתח היא **08:00** בבוקר, אז היינו מייצגים את חלק התאריך כ-1 בינואר שנת 1 ואת השעה כ-**08:00** בבוקר, זה קצת מורכב להשתמש בזה ככה אבל אנחנו בהחלט יכולים לעשות את זה.

אנחנו נראה שזו הדרך הנפוצה ביותר להתמודד עם תאריכים, כאשר אנחנו מתמודדים רק עם חלק מהתאריך, אבל יש לנו אפשרויות אחרות עם **DateOnly** ו-**TimeOnly** שהן תחליפים פוטנציאליים אם אנחנו עוסקים רק בחלק אחד, רק התאריך או רק השעה.

המורכבות של ניהול זמנים והאובייקט DateTime

כאשר מגדירים את משתנה הזמן עם **DateTime.Now**, המערכת שולפת את הזמן המקומי של המכונה שמריצה את הקוד.

בעולם פיתוח התוכנה (ובמיוחד במערכות מבוזרות של **Full Stack**), עבודה עם זמן מקומי היא מתכון ידוע לבאגים קשים בגלל הבדלים באזורי זמן, מעברי שעון קיץ וחורף, ושנים מעוברות.

יש מורכבות לכך שסביבות ייצור אמיתיות דורשות התייחסות קפדנית לאזורי זמן, למרות ש-**DateTime.Now** מספיק לחלוטין לצורכי למידה, בדיקות או אפליקציות מקומיות.

הפרדה בין הנתון בזיכרון לעיצוב התצוגה

עקרון תכנותי קריטי:

ההפרדה בין ה-**Data** (המידע עצמו) לבין ה-**Presentation** (איך שהוא מוצג למשתמש).

כאשר משתמשים בפונקציה **ToString** יחד עם פרמטרים לעיצוב, כמו האותיות **d** או **T**, המשתנה המקורי בזיכרון נשאר ללא שינוי וממשיך לאחסן את התאריך והשעה המדויקים עד רמת המילי שניות.

הפעולה הזו בסך הכל יוצרת "מסנן תצוגה" שמחליט איזה חלק מהמידע יודפס או יועבר הלאה לתצוגת ה-**Frontend**, מבלי לאבד את נתוני המקור השמורים במערכת.

התפתחות השפה ופתרונות מודרניים למגבלות עבר

בעבר, **DateTime** היה כלבוניק – המבנה היחיד ב-**C#** שדרכו ניהלו זמנים.

כתוצאה מכך, נוצר מצב ארכיטקטוני בעייתי:

אם המערכת הייתה צריכה לשמור רק תאריך לידה של משתמש, נאלצו לשמור גם שעה (לרוב **00:00:00** חצות).

אם היו צריכים רק שעת פתיחה של חנות, נאלצו להצמיד לה תאריך פיקטיבי כמו ה-1 בינואר שנת 1. הגישה הזו עדיין נפוצה מאוד בקוד ישן שקיים בתעשייה, אבל יש כלים מודרניים שנכנסו ל-**.NET** (החל מגרסה 6), כמו **DateOnly** ו-**TimeOnly**, שמאפשרים לשמור רק את הנתון הספציפי שצריך. זה חוסך מקום בזיכרון, מונע בלבול והופך את הקוד לנקי ומדויק יותר.

~~~~~

```
DateTime birthday = DateTime.Parse("15/06/1998");

Console.WriteLine(birthday.ToString());
```

## לדוגמה תאריך לידה:

הקוד מציג משתנה תאריך עם הערך **15/06/1998**.

זהו פורמט תאריך תקין, אבל הוא תקין ספציפית עבור הגדרות תאריך של ישראל.  
מכיוון שהמחשב מבוסס על אזור ישראל, ההגדרות המקומיות שלנו (**Locale**) קובעות שהיום מגיע ראשון, לאחר מכן החודש ובסוף השנה.

עם זאת, יש המון מקומות בעולם שבהם המיקום הראשון שייך לחודש והמיקום השני שייך ליום.  
לכן, אם נריץ בדיוק את אותו הקוד במקום אחר שבו ההגדרות המקומיות של המחשב מציינות שהחודש מגיע ראשון, התוכנית תקרוס מכיוון שאין חודש **15**.

זו בדיוק הסיבה בגללה עבודה עם תאריכים ושעות יכולה להפוך למסובכת מאוד.

אנחנו חייבים לדעת מהן ההגדרות המקומיות שאנחנו עובדים איתן.

אם פשוט נתחיל להניח הנחות, אנחנו עלולים להחדיר באגים למערכת.

בפוטנציה, עלולים להיווצר באגים שבהם האפליקציה לא קורסת אלא פשוט ממשיכה לעבוד, אבל לא בצורה שציפינו לה.

הנושא המרכזי הוא הסכנה הגדולה שבהסתמכות עיוורת על הגדרות מערכת ההפעלה המקומית כשעובדים עם תאריכים.

הקוד צפוי לרוץ בסביבות שונות, כמו למשל בשרתי ענן הממוקמים בארצות הברית.

הפעולה של המרת טקסט רגיל לאובייקט של תאריך מסתמכת כברירת מחדל על התרבות של המחשב שמריץ את הקוד באותו רגע.

הבעיה הראשונה היא קריסה חיה של המערכת.

כאשר שרת אמריקאי ינסה לפענח את התאריך **15/06/1998**, הוא ינסה לקרוא את המספר **15** בתור חודש.

מכיוון שבלוח השנה יש רק **12** חודשים, הפעולה תיכשל מיד והתוכנית תקרוס.

הבעיה השנייה, והמסוכנת הרבה יותר, היא יצירת באג שקט.

נדמין שהתאריך שכתבנו הוא **10/05/1998**.

שרת בישראל יקרא את זה כעשירי למאי.

שרת בארצות הברית לא יקרוס בכלל, כי יש חודש **10** ויש יום **5**, אבל הוא יקרא את התאריך בתור החמישי לאוקטובר! הקוד ימשיך לרוץ כרגיל, המידע השגוי יישמר במסד הנתונים שלנו, ולא נקבל שום התראה על כך שהמערכת עובדת עם נתונים לא נכונים.

זהו באג לוגי שקשה מאוד לאתר אחר כך.

כדי להימנע מהמצבים האלה בסביבות פיתוח מתקדמות, אנחנו תמיד נוודא שאנחנו מגדירים למערכת במפורש באיזה פורמט התאריך מגיע, או שנשתמש בפורמט אוניברסלי שאינו משתנה ולא תלוי בשום דרך במיקום הגיאוגרפי של השרת שלנו.

~~~~~

דרך נוספת לעשות זאת:

```
DateTime birthday = DateTime.ParseExact();
```

בעזרת **ParseExact** אנחנו יכולים להיות קצת יותר מדויקים באופן שבו אנחנו מבצעים דברים. אז הדרך היא קודם כל לספק את התאריך שלנו, "**06/11/1998**", אנחנו יודעים שזה תאריך תקין אבל אז אנחנו מספקים לו פורמט.

עם כמה מהאפשרויות מתוך רשימת הפורמטים אנחנו יכולים לציין מחרוזת מותאמת אישית.

למשל d

יום בחודש (**1-2 ספרות**), **1-2** ספרות אומר שאם החודש הוא פברואר שהוא **2**, אנחנו כותבים **2**.

אם נבחר dd

יום בחודש (**2 ספרות**), זה אומר שאם החודש הוא פברואר שהוא **2**, נכתוב **02**. אבל אם נבחר לכתוב:

```
DateTime birthday = DateTime.ParseExact("06/11/1998", "d/M/yyyy");
```

אנחנו בעצם בוחרים בפורמט של "**d/M/yyyy**".

הפורמט הזה נפוץ יותר בשאר העולם, לא בארה"ב, וזה יגרום לקוד שלנו לפרש את התאריך "**06/11/1998**" בתור היום השישי של נובמבר, **1998**.

ה-d

מייצג תאריך קצר.

ה-M

הגדולה מייצגת את החודש.

ה-yyyy

באותיות קטנות מייצגות את השנה (**4 ספרות**). ואנחנו כן חייבים לציין את

```
CultureInfo.InvariantCulture
```

על ידי הוספת השורה

```
using System.Globalization;
```

למעלה.

```
DateTime birthday = DateTime.ParseExact("06/11/1998", "d/M/yyyy", CultureInfo.InvariantCulture);
```

המשמעות היא: אל תדאג לגבי התרבות (**Culture**) שבה אתה נמצא, או האזור המקומי (**Locale**) שבו אתה נמצא, ואיך הם נוהגים לעצב תאריכים שם.

המטרה של ParseExact בעולם האמיתי

כשמקבלים נתונים של תאריך כטקסט מהצד לקוח או ממסד הנתונים, חייבים להמיר אותם לאובייקט מסוג **DateTime** ב-**#C** כדי לעבוד איתם.

אם משתמשים בפונקציה הרגילה של **Parse**, התוכנה מנסה לנחש את התאריך לפי ההגדרות של המחשב שמריץ את הקוד.

זה אומר שעל השרת בישראל התאריך **"06/11"** יובן כ-6 בנובמבר, אבל על שרת בארה"ב הוא יובן כ-11 ביוני, מה שיגרום לבאגים חמורים במערכת.

הפונקציה **ParseExact** פותרת את זה בכך שהיא מכריחה להגדיר בדיוק מוחלט איך בנויה מחרוזת הטקסט.

פירוק מחרוזת הפורמט המותאם אישית

כאשר מגדירים את המחרוזת **"d/M/yyyy"**, הוא בונה תבנית מדויקת.

האות **d** מורה למערכת לחפש את היום בחודש ללא אפס מוביל, כך שהיא תדע לקרוא גם **"6"** וגם **"15"**.

האות **M** הגדולה מורה למערכת לחפש את החודש, שוב ללא **0** מוביל, כדי להבדיל אותה מהאות **m** הקטנה ששמורה ב-**#C** לייצוג דקות.

הרצף **yyyy** מורה למערכת לצפות לשנה בעלת ארבע ספרות בדיוק.

החשיבות של CultureInfo.InvariantCulture

כדי לנתק לחלוטין את הקוד מהגדרות השפה והזמן של השרת עליו הוא רץ, משתמשים באובייקט **InvariantCulture** שמגיע מספריית הגלובליזציה.

ברגע שמעבירים את האובייקט הזה כפרמטר השלישי לפונקציה, בעצם אומרים ל-**#C** להתעלם לחלוטין מאיפה המחשב הזה נמצא בעולם.

להשתמש רק בתבנית המדויקת שסיפקנו לו ובחוקים אוניברסליים ניטרליים כדי להבין את התאריך.

זוהי פרקטיקת חובה בקוד מודרני ואמין כדי להבטיח שהתוכנה תתנהג תמיד אותו הדבר, ללא תלות בסביבת הריצה.

~~~~~

ה-**MMMM**

חודש מלא.

**ה-dd**

יום ב-2 ספרות.

**ה-yyyy**

שנה ב-4 ספרות.

**ה-hh:mm**

שעון 12 שעות (2 ספרות), דקות (2 ספרות).

**ה-tt**

מלא AM/PM

**ה-zzz**

עבור אזור הזמן, הסטת שעות ודקות מ-UTC  
(זמן אוניברסלי מתואם).

## המטרה הכללית

השורה הזו לוקחת משתנה של תאריך ושעה (כמו **המשתנה today**) וממירה אותו למחרוזת **string** בפורמט מאוד ספציפי ומוגדר מראש.  
ההמרה מתבצעת תוך התעלמות מהגדרות השפה או האזור הספציפיות של המחשב או השרת שעליו רץ הקוד.

## עיצוב התאריך והשעה מותאם אישית

הפונקציה **ToString** מקבלת כאן מחרוזת שמהווה תבנית (**Custom Format String**).  
כל צירוף אותיות בתבנית מורה לקומפיילר של **#C** כיצד להציג חלק אחר של התאריך.  
האותיות **MMMM** ידפיסו את שם החודש המלא, למשל יולי (**July**).  
האותיות **dd** ידפיסו את היום בחודש תמיד עם שתי ספרות, כך שאם מדובר ביום החמישי בחודש, יודפס **05**.

האותיות **yyyy** ידפיסו את השנה במלואה עם ארבע ספרות, למשל **2026**.  
האותיות **hh:mm** ידפיסו את השעה בפורמט של **12** שעות (**כלומר 01 עד 12**) ואת הדקות, שתיהן בשתי ספרות.  
האותיות **tt** יוסיפו את הציון **AM** או **PM** כדי להבדיל בין שעות הבוקר לשעות הערב.  
האותיות **zzz** יוסיפו את קיזוז אזור הזמן המקומי ביחס לזמן האוניברסלי, למשל פלוס **03:00** עבור שעון ישראל.  
**הפלט הסופי של כל המחרוזות יראה בערך כך: July 10, 2026 04:01 AM +03:00**

## השימוש ב-CultureInfo.InvariantCulture

זהו פרט קריטי לפיתוח מערכות יציבות ואמינות (**Best Practice** חובה בסביבות פרודקשן).  
הפרמטר השני, **CultureInfo.InvariantCulture**, קובע שעיצוב המחרוזת לא יושפע מההגדרות המקומיות של מערכת ההפעלה.  
אם לא היינו משתמשים בזה, והקוד היה רץ על שרת שמוגדר לעברית או לצרפתית, שמות החודשים או כיוון התאריך היו משתנים בהתאם.  
השימוש ב-**InvariantCulture** מבטיח שהפלט יהיה תמיד אחיד, אוניברסלי וסטנדרטי באנגלית.  
זה מונע באגים קריטיים כאשר מעבירים נתונים בין שרתים, שומרים תאריכים למסד נתונים, או משדרים מידע ל-**.Frontend**.

~~~~~

```
DateTime today = DateTime.UtcNow;  
DateTime.Now  
DateTime.UtcNow
```

כשאני אומר **Now**, הכוונה היא לזמן המקומי.
אנחנו חייבים לחשוב איפה הקוד שלנו הולך לרוץ, כי לעיתים קרובות אנשים חושבים שהם יכולים להשתמש ב-**Now** בגלל שהקוד שלהם תמיד ירוץ איפה שהם נמצאים.
ובכן... אולי.
כי אם נגיד שעכשיו הגיע הזמן לעבור לענן ונריץ את האפליקציה שלנו בענן, היא עשויה לרוץ באזור זמן שונה כי השרת עשוי להיות באזור זמן שונה משלנו.
אנחנו עשויים להגיד שתמיד נשים את השרת באותו אזור זמן כמונו, ולכן **Now** עדיין יעבוד.
ובכן... אבל מה קורה כשהשרת נופל... כנראה שיש לנו גיבוי איפשהו ובדרך כלל הגיבוי הזה נמצא באזור זמן אחר.
אז אם אנחנו שומרים לוגים לגבי מתי דברים קרו ומשתמשים ב-**DateTime.Now** מבלי לציין או לשמור את הסטיית זמן (**offset**), זו יכולה להיות בעיה.
אז יש אפשרות נוספת והיא שאנחנו יכולים להשתמש ב-**DateTime.UtcNow**.
זהו זמן אוניברסלי מתואם (**UTC**), זהו זמן גריניץ', זהו הזמן שכל הזמנים שלנו מבוססים עליו.

אז זהו זמן ה-UTC, אנחנו יכולים לרשום הכל בזמן UTC כי אז זה לא משנה אם זה רץ בחוף המזרחי או בחוף המערבי או באירופה, זה תמיד יהיה אותו הזמן ביחס לכל שאר הזמנים.

עם זאת, אנחנו מאבדים את היכולת לדעת מה הייתה סטיית הזמן עבור כל מה שנוצר, ואנחנו צריכים להחליט אם זה בסדר או לא.

אז יש זמן UTC וזמן מקומי, ואנחנו מאבדים מידע כשאנחנו לוכדים רק את זמן ה-UTC, אבל אנחנו הופכים את השוואת התאריכים לקלה יותר.

הסבר למפתחי תוכנה בסביבת ענן

הבעיה עם שימוש בזמן מקומי

כאשר אנחנו כותבים קוד למערכות מודרניות, לרוב הן ירוצו על שרתים בענן.

אם נשתמש בפקודה של זמן מקומי, המערכת תיקח את השעה המקומית של השרת הפיזי שעליו היא יושבת באותו רגע.

הבעיה מתחילה כאשר האפליקציה מתרחבת לשרתים באזורים גיאוגרפיים שונים, או כאשר יש נפילת שרת והקוד עובר לרוץ אוטומטית על שרת גיבוי שנמצא במדינה אחרת לגמרי.

במצב כזה, רישומי המערכת שלנו יציגו זמנים לא עקביים, ויהיה בלתי אפשרי לנתח לוגים או להבין מתי באמת התרחשה שגיאה או מתי משתמש ביצע פעולה.

הפתרון של זמן אוניברסלי מתואם

כדי לפתור את הבעיה הזו, הסטנדרט בתעשייה הוא שימוש בזמן גלובלי אחיד.

השעה הזו אינה מושפעת ממיקום השרת בעולם או משינויי שעון קיץ וחורף.

היתרון הגדול הוא ששמירת נתונים בדרך זו מאפשרת למערכת כולה לדבר באותה שפת זמנים, מה שהופך חישובים, סינונים והשוואות בין תאריכים במסד הנתונים למשימה פשוטה ומדויקת מאוד, ללא תלות במיקום הפיזי של הקוד באותו רגע.

המחיר של אחידות הנתונים

החיסרון המרכזי הוא אובדן ההקשר המקומי.

ברגע ששומרים רק את הזמן הגלובלי, אנחנו מאבדים את המידע לגבי מה הייתה השעה הספציפית באזור הזמן המקורי שבו התבצעה הפעולה.

כמפתחים, אנחנו צריכים להבין את ההשלכות של הוויתור הזה על המידע המקומי, ולהחליט ארכיטקטונית האם עבור המערכת שאנחנו בונים האחידות הגלובלית היא הדבר החשוב ביותר, או שיש צורך לשמור בנפרד גם את נתוני הפער מהשעון המקומי כדי לא לאבד את המידע הזה.